



Universitat
Oberta
de Catalunya

(HHTMLCSS2) Herramientas HTML y CSS II
PEC 1 - Workflow front-end moderno y
metodologías CSS

Marc Turu Roca

Profesor: Julián Alarte Aleixandre

Fecha de entrega: 30/3/2026

Universitat Oberta de Catalunya (UOC)

Tabla de contenidos

1. Introducción	2
2. Configuración inicial	2
2.1 Módulo 1: Un workflow moderno de desarrollo front-end	2
2.2 Módulo 2: Guías de estilo y metodologías CSS	3
3. Desarrollo de la aplicación	5
3.1 Idea y diseño	5
3.2 Estructuración de los archivos	6
3.3 Implementación	7
3.3.1 Componentes	7
3.3.2 Estilos	7
3.3.3 Scripts	10
4. Conclusión	11
5. Anexos	12

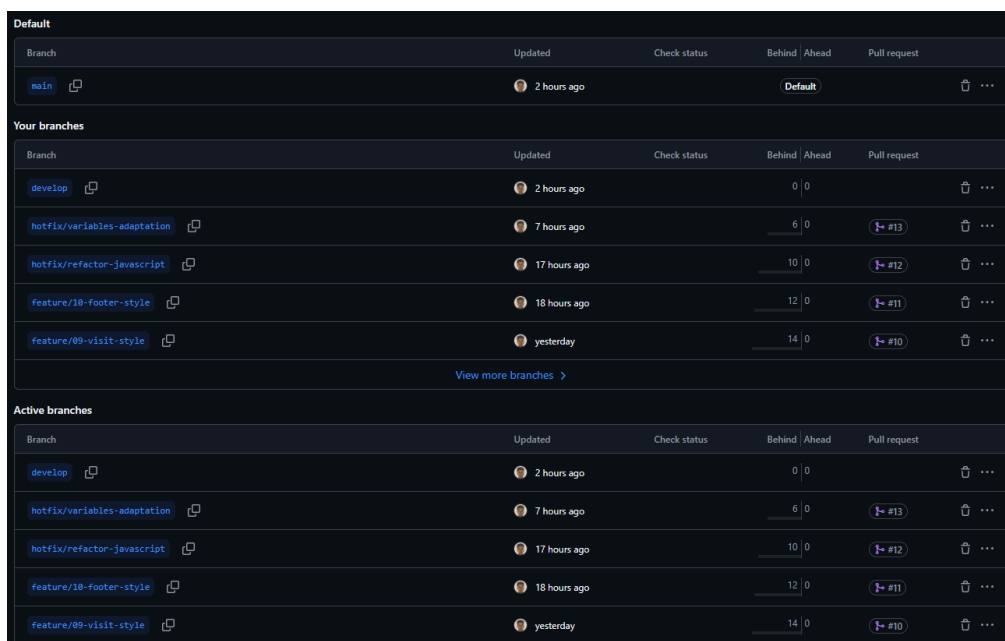
1. Introducción

Esta práctica ha consistido en el desarrollo de un sitio web de una sola página, partiendo de [UOC Boilerplate](#) y aplicando conceptos propios de workflows modernos de desarrollo front-end y guías de estilo y metodologías CSS, entre otros.

2. Configuración inicial

2.1 Módulo 1: Un workflow moderno de desarrollo front-end

Para esta primera parte, a parte de actualizar la version de Node para poder utilizar herramientas modernas, se hizo un `git clone` <https://github.com/uoc-advanced-html-css/uoc-boilerplate.git> para poder disponer de la plantilla inicial sobre la cual trabajar. Además, se creó un repositorio nuevo <https://github.com/marcturu/kauai-hawaii> para poder llevar a cabo un control de versionado del código desarrollado. En este repositorio se mantuvo la rama **main** para presentar los “lanzamientos” de las versiones finales, se creó una rama **develop** para poder juntar las diferentes **features/xx-nombre-feature** y se reservaron las ramas **hotfix/nombre-hotfix** para resolver problemas que surgieran en el código a lo largo del proyecto.



Branch	Updated	Check status	Behind	Ahead	Pull request
main	2 hours ago			Default	...

Branch	Updated	Check status	Behind	Ahead	Pull request
develop	2 hours ago		0	0	...
hotfix/variables-adaptation	7 hours ago		6	0	#13
hotfix/refactor-javascript	17 hours ago		10	0	#12
feature/10-footer-style	18 hours ago		12	0	#11
feature/09-visit-style	yesterday		14	0	#10

Branch	Updated	Check status	Behind	Ahead	Pull request
develop	2 hours ago		0	0	...
hotfix/variables-adaptation	7 hours ago		6	0	#13
hotfix/refactor-javascript	17 hours ago		10	0	#12
feature/10-footer-style	18 hours ago		12	0	#11
feature/09-visit-style	yesterday		14	0	#10

Fig. 1 - Ramas de GitHub

Seguidamente, se hizo un `npm install --save @fortawesome/fontawesome-free` y `@import "npm:@fortawesome/fontawesome-free/css/all.css";` en `src/assets/styles/_dependencies.scss` para poder usar los iconos de Font Awesome.

Para terminar con esta primera parte, y después de realizar algunos cambios de prueba en el código y probarlos con `npm run dev`, se ejecutó el comando `npm run build` para poder crear la carpeta `dist/` con el código listo para producción, el cual, una vez creado el entorno de despliegue en Netlify y configurado con la cuenta GitHub y los parámetros necesarios, serviría para servir la web de una manera más optimizada.

2.2 Módulo 2: Guías de estilo y metodologías CSS

Para esta segunda parte, se estudiaron las diferentes guías de estilo (siendo [CSS Guidelines](#) la adoptada) y metodologías CSS y se instalaron y configuraron las dependencias que serían necesarias para la web.

La metodologías que se decidió usar fue **ITCSS** (para estructurar las carpetas y jerarquía de capas en SCSS en `src/assets/styles/`), **BEM** (en `components/` para usar nombres de clases consistentes y predecibles) y **OOCSS** (en `objects/` para definir bloques reutilizables con su propia estructura).

Esta combinación de metodologías permitió seguir un plan de desarrollo de estilos ordenado y reutilizable, perfecto para escalar en cualquier momento.

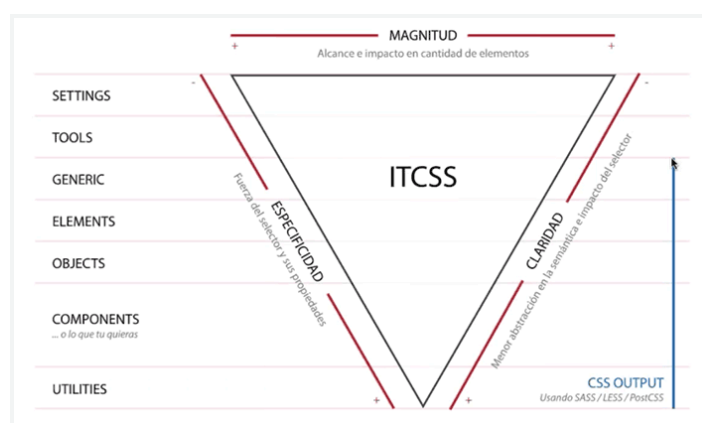


Fig. 2 - Representación visual de la metodología ITCSS

También se instaló **Stylelint** consultando las `peerDependencies` de cada dependencia a instalar para consultar su compatibilidad:

```
Marc T@DESKTOP-CT81S8L MINGW64 ~/Desktop/test
• $ npm info stylelint-scss peerDependencies
{ stylelint: '^16.8.2 || ^17.0.0' }

Marc T@DESKTOP-CT81S8L MINGW64 ~/Desktop/test
• $ npm info stylelint-config-recommended-scss peerDependencies
{ postcss: '^8.3.3', stylelint: '^17.0.0' }

Marc T@DESKTOP-CT81S8L MINGW64 ~/Desktop/test
• $ npm info stylelint-config-prettier-scss peerDependencies
{ stylelint: '>=15.0.0' }

Marc T@DESKTOP-CT81S8L MINGW64 ~/Desktop/test
• $ npm info stylelint-scss peerDependencies
{ stylelint: '^16.8.2 || ^17.0.0' }
```

Fig. 3 - `npm info <dependencia> peerDependencies` para cada dependencia de Stylelint a instalar

Finalmente se optó por instalar las siguientes versiones:

```
npm install --save-dev1 stylelint@17 stylelint-scss@7
stylelint-config-recommended-scss@17 stylelint-config-prettier-scss
```

Para poder personalizar las reglas a aplicar dentro del Stylelint, se configuró el `.stylelintrc` con valores como por ejemplo:

```
"selector-class-pattern":
"^([a-z])([a-z0-9-]+)?(__([a-z0-9-]+-?)+)?(--([a-z0-9-]+-?)+){0,2}$"
```

, el cual adapta la expresión regular propuesta en el enunciado eliminando el carácter inicial, ya que Stylelint evalúa los nombres de clase sin el selector `'.'`.

Se añadió `npm install --save-dev`

```
"stylelint": "stylelint src/**/*.scss"
"build": "npm-run-all clean stylelint parcel:build",
```

en el `package.json` para que, cada vez que se quisiera actualizar el código de producción de la carpeta `dist/`, se llevaran a cambio las comprobaciones de estilos con `npm run stylelint`.

```
marct@LAPTOP-OIR973V9 MINGW64 ~/Desktop/UOC-EHTMLCSS2-PEC1-MarcTuruRoca (develop)
$ npm run stylelint

> uoc-boilerplate@3.13.0 stylelint
> stylelint src/**/*.scss
```

Fig. 4 - Estilos pasados correctamente con `npm run stylelint`

¹ Se añadió `--save-dev` para que las dependencias apareciesen en el `devDependencies` y se usaran en el entorno de desarrollo.

3. Desarrollo de la aplicación

3.1 Idea y diseño

Debido a que el proyecto consistía en desarrollar una página web sobre un paraje que se considerase interesante y que valiera la pena visitar, aparte de que el contenido fuese coherente, quería hacerlo sobre algún sitio especial, que me permitiese usar recursos multimedia de calidad y fuese divertido de implementar. Es por eso que, debido a mi afición por la geografía y descubrir sitios nuevos, y la nostalgia de una saga de películas que me gustaba mucho, decidí hacerlo sobre **Kaua'i**, la isla hawaiana que sirvió como localización (entre otras) del rodaje de Jurassic Park.

La intención era darle un toque moderno a la página pero que cualquier usuario pudiese usarla independientemente de sus conocimientos tecnológicos. Debido a esto, se incorporaron las siguientes dependencias:

- **Lenis** (^1.3.21). *Scroll* suave y fluido
- **Leaflet** (^1.9.4). Mapa interactivo
- **Swiper** (^12.1.3). Carrusel táctil
- **AOS** (^2.3.4). Animaciones al hacer *scroll*, ya que implementándolas desde `_helpers.scss` surgían errores de UX.

La página web se estructuró utilizando **PostHTML** a través de las inclusiones de diferentes componentes:

1. **Header**. Logo y barra de navegación con secciones
2. **Hero**. Vídeo a pantalla completa
3. **The Garden Isle**. Introducción a la isla y su relación con Jurassic Park
4. **Filming Locations**. Cuatro localizaciones reales donde se rodó Jurassic Park, con enlaces a Google Maps
5. **Interactive Map**. Mapa interactivo con Leaflet y marcadores personalizados para cada localización
6. **Photo Gallery**. Carrusel táctil con Swiper y fotografías de Kaua'i
7. **Plan Your Trip**. Información práctica para viajar (cómo llegar, mejor época, cómo moverse)
8. **Footer**. Créditos del proyecto

Además, se optó por seguir un estilo de diseño **mobile-first**, es decir, programando primero la versión para dispositivos móviles y adaptándola a resoluciones mayores.

3.2 Estructuración de los archivos

```
src/
├─ index.html
├─ views/                # Parciales PostHTML
│  ├─ header.html
│  ├─ hero.html
│  ├─ about.html
│  ├─ locations.html
│  ├─ map.html
│  ├─ gallery.html
│  ├─ visit.html
│  └─ footer.html
├─ assets/
│  ├─ fonts/             # Fuentes autoalojadas (Unbounded, DM Sans)
│  ├─ images/            # Imágenes optimizadas en WebP
│  ├─ videos/            # Vídeo del hero (MP4)
│  ├─ scripts/
│  │  ├─ main.js
│  │  └─ modules/        # Módulos JS (lenis, aos, swiper, leaflet, header)
│  └─ styles/
│     ├─ main.scss
│     ├─ _dependencies.scss
│     ├─ settings/        # Variables, fuentes
│     ├─ tools/           # Funciones, mixins
│     ├─ generic/         # Reset
│     ├─ elements/        # Estilos base
│     ├─ objects/         # Objetos OOCSS
│     ├─ components/      # Componentes BEM
│     └─ utilities/       # Clases de utilidad
```

Fig. 5 - Estructura de los archivos del proyecto (disponible en el README.md)

Se separaron:

- Las diferentes secciones del **index.html** (header.html,...)
- Los diferentes recursos de los **assets/** (fonts,...)
- Las diferentes imágenes de las **images/** (locations,...)
- Los diferentes scripts de los **modules/** (aos.js,...)
- Los diferentes estilos de los ITCSS **styles/** (settings,...)

De esta manera, los archivos principales como *index.html*, *main.js* y *main.scss* (entre otros), tenían el código justo y necesario para poder importar y usar los demás componentes y funcionalidades, respectivamente.

También se optó por mejorar la resolución de las imágenes *.webp* con [Squoosh](#) y el vídeo con [HandBrake](#).

3.3 Implementación

3.3.1 Componentes

Con la estructura compuesta, se pasó a implementar los componentes de cada sección de manera secuencial respecto a la distribución vertical de la página web. Para ello, se usaron plantillas HTML con clases BEM para los elementos de las secciones de cada apartado.

El *head* principal contenía información como por ejemplo:

```
<link rel="icon" type="image/x-icon"
href="./assets/images/favicon/favicon.ico" />
<link rel="stylesheet" type="text/css" href="./assets/styles/main.scss" />
<title>Kaua'i - Where Jurassic Park Was Born</title>
```

además del uso de PostHTML con:

```
<include src="./views/header.html"></include>
```

uso activo de dependencias en las vistas como:

```
data-aos="fade-right" data-aos-delay="100"
```

e inclusión de estilos en línea en las imágenes para reservar espacio y mejorar la carga de estas:

```
width="1200"
height="1000"
loading="lazy"
decoding="async"
```

Se hizo uso de una variedad de etiquetas HTML, como `header` para la parte del encabezado con la barra de navegación, `footer` para la parte del pie de página, `section` para secciones de cada apartado, `div` para contenedores sin una función específica, `h1,...` para jerarquía de títulos, `p` para párrafos, `nav` para la barra de navegación, `ul` para listas desordenadas y `li` para elementos de estas listas, `a` para enlaces, `img` para imágenes y `button` para botones interactivos.

3.3.2 Estilos

Los estilos siguieron la metodología ITCSS mostrada en la figura 5, adaptándose a los OOCSS declarados en `src/assets/styles/objects` y los BEM de las plantillas HTML.

En el archivo `main.scss`, se declaraban los diferentes archivos que formaban parte del grupo de estilos y que seguían la sintaxis SCSS de Sass.

Settings. Se encontraban los archivos `_fonts.scss` y `_variables.scss`.

En ellos, se declaraban las familias de fuentes y variables a utilizar, respectivamente:

```
@font-face {
  font-family: 'Unbounded';
  src: url('../fonts/unbounded-v12-latin-regular.woff2')
format('woff2');
  font-weight: 400;
  font-display: swap;
}
...
```

```
$color-neon: #0f8;
$spacing-xs: 0.5rem;
...
```

Tools. Se encontraban los archivos `_functions.scss` y `_mixins.scss`.

En ellos, se declaraban las funciones y adaptaciones de resoluciones a utilizar, respectivamente:

```
@function color-alpha($color, $alpha) {
  @return rgba($color, $alpha);
}
...
```

```
@mixin respond-to($bp) {
  @if $bp == sm {
    @media (min-width: v.$bp-sm) { @content; }
  } @else if $bp == md {
    @media (min-width: v.$bp-md) { @content; }
  } @else if $bp == lg {
    @media (min-width: v.$bp-lg) { @content; }
  } @else if $bp == xl {
    @media (min-width: v.$bp-xl) { @content; }
  }
}
...
```

Generic. Se encontraba el archivo `_reset.scss`.

En este, se reiniciaban algunos valores de las etiquetas propias de HTML:

```
body, h1, h2, h3, h4, h5, h6, p, figure, blockquote, dl, dd {
  margin: 0;
  padding: 0;
}
...
```

Elements. Se encontraba el archivo `_basescss`.

En este, se establecían valores generales para algunas etiquetas propias de HTML:

```
main {
  padding-top: v.$spacing-lg;
}
...
```

Objects. Se encontraban los archivos `_container.scss` y `_section.scss`.

En ellos, se declaraban estilos OOCSS para definir bloques reutilizables con su propia estructura, respectivamente:

```
.o-container {
  width: 100%;
  max-width: v.$container-max-width;
  margin-inline: auto;
  padding-inline: v.$container-padding;
}
```

```
.o-section {
  padding-block: v.$spacing-lg;
}
```

Components. Se encontraban los archivos `_header.scss`, `_hero.scss`, `_about.scss`, `_locations.scss`, `_map.scss`, `_gallery.scss`, `visit.scss` y `footer.scss`.

En ellos, se declaraban los estilos específicos para cada vista HTML.

Aunque en otros archivos también, en estos se veía más claro el uso de sintaxis SCSS de Sass con variables, parciales, funciones, anidados y BEM:

```
.about {
  padding-block: v.$spacing-lg;
  background-color: v.$color-background;

  &__container {
    display: flex;
    flex-direction: column;
    gap: v.$spacing-md;
  }

  &__text {
    display: flex;
    flex-direction: column;
    gap: v.$spacing-sm;
  }
}
```

y el uso de distintas herramientas ITCSS:

```
@use "../tools/mixins" as m;

@include m.respond-to(md) {
  font-size: 2.5rem;
}

padding-block: v.$spacing-lg;

text-shadow: 0 0 8px f.color-alpha(v.$color-neon, 0.4);
```

3.3.3 Scripts

Los scripts sirvieron como complemento para poder añadir funcionalidades a la página web y estilizar dinámicamente elementos de los HTML y SCSS.

En el *header.js*, se llevaron a cabo prácticas como:

```
const header = document.querySelector('.header')
const mobileNav = document.querySelector('.header__mobile-nav')
const updateHeaderOpacity = () => {
  const isMenuOpen =
mobileNav.classList.contains('header__mobile-nav--open')
  if (isMenuOpen) return // Si el menu está abierto, no se cambia la
opacidad (tiene que ser 1)
  header.style.setProperty('--header-opacity', getScrollOpacity())
}
```

para actualizar la opacidad del header según la posición del *scroll*, entre otras.

En los demás archivos javascript, se instanciaron y configuraron según una inicialización las dependencias a utilizar, por ejemplo:

```
import Lenis from 'lenis';

export const lenis = new Lenis({
  duration: 0.9,
  easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)),
  anchors: true,
})

export function initLenis() {
  function raf(time) {
    lenis.raf(time);
    requestAnimationFrame(raf);
  }
  requestAnimationFrame(raf);
}
```

4. Conclusión

Este proyecto ha resultado ser una actividad muy útil y enriquecedora. Desde las primeras instalaciones donde surgieron algunas dudas y problemas respecto a las versiones de las dependencias, hasta el despliegue final a producción, ha sido un proceso donde, a parte de ser entretenido, he aprendido muchas tecnologías y dependencias nuevas a la vez que entendido mejor las que ya conocía.

El stack de Parcel como bundler, SCSS de Sass para los estilos (con Stylelint y PostCSS para reglas y compatibilidad), PostHTML para dividir las vistas, las dependencias utilizando Javascript en gran parte y la publicación de la web en internet a través de Netlify ha sido una estructura muy correcta, cada componente útil en su medida.

En cuanto a aspectos de mejora, en la parte de distribución del tiempo podría haber planificado el desarrollo de la práctica de mejor manera y explorado más dependencias. Aún así, estoy satisfecho y contento con el proyecto.

5. Annexos

1. **Aplicación web.** Kaua'i - Where Jurassic Park Was Born (**Netlify**)

<https://kauai-hawaii.netlify.app/>

2. **Vídeo explicación + capturas de pantalla (Drive):**

<https://drive.google.com/file/d/1cX-MurqL7Eu9t1v9-fAe1okrWuosLZRK/view?usp=sharing>

https://drive.google.com/drive/folders/1QOwH3Wz4_DURuAlp0XHNC6wRQyOR9diH?usp=drive_link

3. **Repositorio con el código (GitHub):**

<https://github.com/marcturu/kauai-hawaii>